

Roles y Control de acceso

Contents

1	Policies and Gates	1
1.1	Entendiendo los Gates	2
1.1.1	Definiendo Gates	2
1.1.2	Autorizando Acciones Usando Gates	2
1.2	Entendiendo las Policies	3
1.2.1	Generando Policies	3
1.2.2	Escribiendo Policies	3
1.2.3	Registrando Policies	3
1.2.4	Autorizando Acciones Usando Policies	3
1.3	Mejores Prácticas	4
2	Manejo de roles y control de acceso directo	4
2.1	Paso 1: Diseñar el Modelo de Datos	4
2.2	Paso 2: Crear Modelos y Relaciones	4
2.3	Paso 3: Sembrar la Base de Datos con Roles y Permisos	4
2.4	Paso 4: Crear Middleware para Verificar Permisos	5
2.5	Paso 5: Verificar Permisos en los Controladores o Rutas	5
2.6	Paso 6: Crear Métodos de Ayuda en el Modelo de Usuario	5
2.7	Conclusión	5
3	Roles y control de acceso con Laravel Breeze	6
3.1	Paso 1: Instalar Laravel Breeze	6
3.2	Paso 2: Diseñar el Modelo de Datos para Roles y Permisos	6
3.3	Paso 3: Crear Modelos y Establecer Relaciones	6
3.4	Paso 4: Middleware para Verificar Roles y Permisos	6
3.5	Paso 5: Usar el Middleware en Rutas	7
3.6	Paso 6: Gestión de Roles y Permisos en la UI	7
3.7	Conclusión	7

1 Policies and Gates

Laravel ofrece una manera poderosa y flexible de manejar la autorización con dos características principales: Gates (Puertas) y Policies (Políticas). Los Gates proporcionan un enfoque simple basado en closures para la autorización, mientras que las Policies, como clases, organizan la lógica

de autorización alrededor de un modelo o recurso particular. Aquí tienes una guía para entender y usar los Gates y Policies de Laravel en tus aplicaciones.

1.1 Entendiendo los Gates

Los Gates son closures que determinan si un usuario está autorizado para realizar una acción dada. Se definen en el `AuthServiceProvider` y se utilizan típicamente para acciones que no requieren una instancia de modelo.

1.1.1 Definiendo Gates

Los Gates se definen en el método `boot` de tu `AuthServiceProvider`. Un gate recibe una instancia de usuario y cualquier argumento adicional relevante para la decisión de autorización.

```
use Illuminate\Support\Facades\Gate;

/**
 * Registra cualquier servicio de autenticación/autorización.
 *
 * @return void
 */
public function boot()
{
    $this->registerPolicies();

    Gate::define('edit-settings', function ($user) {
        return $user->isAdmin;
    });
}
```

1.1.2 Autorizando Acciones Usando Gates

Puedes verificar si un gate permite a un usuario realizar una acción usando varios métodos como `allows`, `denies`, o las directivas de Blade `@can` y `@cannot`.

```
if (Gate::allows('edit-settings', $user)) {
    // El usuario puede editar configuraciones
}

if (Gate::denies('edit-settings', $user)) {
    // El usuario no puede editar configuraciones
}
```

En las plantillas Blade, puedes usar:

```
@can('edit-settings')
    <!-- Mostrar formulario de edición de configuraciones -->
@endcan

@cannot('edit-settings')
    <!-- Mostrar un mensaje de prohibido -->
@endcannot
```

1.2 Entendiendo las Policies

Las Policies son clases que organizan la lógica de autorización para un modelo o recurso particular. Cada policy es responsable de determinar si un usuario puede realizar una acción sobre un recurso.

1.2.1 Generando Policies

Puedes generar una policy usando la herramienta de línea de comandos Artisan. Por ejemplo, para crear una policy para un modelo `Post`:

```
php artisan make:policy PostPolicy --model=Post
```

Este comando generará una clase `PostPolicy` en el directorio `app/Policies`.

1.2.2 Escribiendo Policies

Las Policies definen métodos que corresponden a las diversas acciones que un usuario podría realizar en una instancia del modelo, tales como ver, crear, actualizar o eliminar.

```
/**
 * Determina si el post dado puede ser actualizado por el usuario.
 *
 * @param \App\Models\User $user
 * @param \App\Models\Post $post
 * @return bool
 */
public function update(User $user, Post $post)
{
    return $user->id === $post->user_id;
}
```

1.2.3 Registrando Policies

Las Policies necesitan ser registradas dentro del `AuthServiceProvider`. Esto se hace típicamente en el array de propiedad `policies`.

```
protected $policies = [
    'App\Models\Post' => 'App\Policies\PostPolicy',
];
```

1.2.4 Autorizando Acciones Usando Policies

Puedes verificar si una policy autoriza una acción de manera similar a los gates. Usa métodos como `can` en una instancia de usuario o las directivas de Blade `@can` y `@cannot`, especificando la acción y el modelo relevante.

```
if ($user->can('update', $post)) {
    // El usuario puede actualizar el post
}
```

En las plantillas Blade, especifica la instancia del modelo:

```
@can('update', $post)
    <!-- Mostrar formulario de actualización del post -->
@endcan
```

1.3 Mejores Prácticas

- **Usa Gates para acciones no relacionadas con modelos**, como ver un panel de administración.
- **Usa Policies para acciones relacionadas con modelos**, asegurando que tu lógica de autorización esté organizada y asociada directamente con operaciones del modelo.
- **Aprovecha el auto-descubrimiento de policy** en Laravel, que descubre automáticamente policies siempre que se sigan las convenciones de nomenclatura de modelo y policy.

Al aprovechar Gates y Policies, puedes mantener un enfoque limpio y organizado para manejar la autorización dentro de tus aplicaciones Laravel, manteniendo tu código seguro y manejable.

2 Manejo de roles y control de acceso directo

Para manejar la autorización basada en roles y permisos directamente desde la base de datos, sin utilizar Gates y Policies de Laravel, puedes seguir una estrategia que involucre la creación de tablas adicionales para roles y permisos, y luego verificar estos roles y permisos en tu aplicación. A continuación, se presenta una guía alternativa para implementar este sistema de autorización:

2.1 Paso 1: Diseñar el Modelo de Datos

Primero, necesitas diseñar tu base de datos para almacenar roles y permisos:

- **Tabla `roles`**: Almacena los roles de los usuarios (por ejemplo, Admin, Editor, Usuario Regular).
- **Tabla `permissions`**: Almacena los permisos específicos (por ejemplo, 'edit post', 'delete post').
- **Tabla `role_user`**: Una tabla de pivote que relaciona los usuarios con sus roles.
- **Tabla `permission_role`**: Una tabla de pivote que relaciona los roles con sus permisos.

2.2 Paso 2: Crear Modelos y Relaciones

Después de crear las tablas, define los modelos `Role` y `Permission` en Laravel, y establece las relaciones adecuadas:

- Un `User` puede tener muchos `Roles`.
- Un `Role` puede tener muchos `Permissions`.

2.3 Paso 3: Sembrar la Base de Datos con Roles y Permisos

Utiliza seeders para poblar tus tablas de roles y permisos con los valores iniciales.

2.4 Paso 4: Crear Middleware para Verificar Permisos

Puedes crear un middleware que verifique si un usuario tiene un permiso específico antes de permitirle acceder a una ruta:

```
php artisan make:middleware CheckPermission
```

En este middleware, verifica si el usuario tiene el permiso necesario para realizar la acción:

```
public function handle($request, Closure $next, $permission)
{
    if (!auth()->user() || !auth()->user()->hasPermission($permission)) {
        // Redireccionar o lanzar una excepción
        abort(403, 'No estás autorizado para realizar esta acción.');
```

2.5 Paso 5: Verificar Permisos en los Controladores o Rutas

Usa el middleware en tus rutas o controladores para proteger el acceso basado en permisos:

```
Route::get('/post/edit/{id}', 'PostController@edit')
    ->middleware('checkPermission:edit post');
```

2.6 Paso 6: Crear Métodos de Ayuda en el Modelo de Usuario

En tu modelo `User`, puedes agregar métodos como `hasRole()` o `hasPermission()` para verificar los roles y permisos del usuario:

```
public function hasRole($role)
{
    return $this->roles->contains('name', $role);
}

public function hasPermission($permission)
{
    return $this->roles->pluck('permissions')
        ->flatten()->contains('name', $permission);
}
```

2.7 Conclusión

Este enfoque te permite tener un control fino sobre quién puede hacer qué dentro de tu aplicación, basándote completamente en la configuración almacenada en tu base de datos. La ventaja de este método es la flexibilidad para cambiar roles y permisos sin necesidad de ajustar el código, simplemente actualizando las entradas en la base de datos.

Sin embargo, es importante notar que este enfoque requiere una cuidadosa gestión de la base de datos y posiblemente una interfaz de usuario para administrar roles y permisos, lo que podría aumentar la complejidad de tu aplicación. Además, debes asegurarte de que tu sistema de roles y permisos esté bien diseñado para evitar brechas de seguridad.

3 Roles y control de acceso con Laravel Breeze

Para integrar un sistema de roles y permisos en una aplicación Laravel que utiliza Laravel Breeze para la autenticación, puedes seguir una serie de pasos que te permitirán verificar los roles y permisos de los usuarios. Aunque Laravel Breeze no proporciona directamente un sistema de gestión de roles y permisos, puedes extender tu aplicación con estas funcionalidades. Aquí te muestro cómo hacerlo:

3.1 Paso 1: Instalar Laravel Breeze

Si aún no has instalado Laravel Breeze, comienza por hacerlo para configurar la autenticación básica:

```
composer require laravel/breeze --dev
php artisan breeze:install
npm install && npm run dev
php artisan migrate
```

3.2 Paso 2: Diseñar el Modelo de Datos para Roles y Permisos

Necesitarás tablas adicionales para manejar roles y permisos:

- **roles:** Almacena roles como Admin, Editor, etc.
- **permissions:** Almacena permisos como 'edit post', 'view post', etc.
- **role_user:** Tabla de pivote para relacionar usuarios con roles.
- **permission_role:** Tabla de pivote para relacionar roles con permisos.

Puedes crear estas tablas usando migraciones de Laravel.

3.3 Paso 3: Crear Modelos y Establecer Relaciones

Crea modelos `Role` y `Permission` y define las relaciones adecuadas en los modelos `User`, `Role`, y `Permission`. Por ejemplo, en tu modelo `User`, podrías tener:

```
public function roles()
{
    return $this->belongsToMany(Role::class);
}

public function hasRole($role)
{
    return $this->roles()->where('name', $role)->exists();
}
```

Y métodos similares en `Role` y `Permission` para gestionar las relaciones.

3.4 Paso 4: Middleware para Verificar Roles y Permisos

Crea un middleware que verifique si el usuario tiene el rol o permiso necesario para acceder a una ruta:

```
php artisan make:middleware CheckRole
```

En el middleware, puedes verificar el rol del usuario:

```
public function handle($request, Closure $next, $role)
{
    if (!$request->user() || !$request->user()->hasRole($role)) {
        abort(403);
    }

    return $next($request);
}
```

Registra tu middleware en `app/Http/Kernel.php` para poder usarlo en tus rutas.

3.5 Paso 5: Usar el Middleware en Rutas

Protege las rutas especificando el middleware y el rol necesario:

```
Route::middleware(['auth', 'checkRole:admin'])->group(function () {
    Route::get('/admin', function () {
        // Solo accesible por usuarios con rol 'admin'
    });
});
```

3.6 Paso 6: Gestión de Roles y Permisos en la UI

Para gestionar roles y permisos desde la interfaz de usuario, podrías crear vistas y controladores que permitan asignar roles a los usuarios y gestionar permisos. Laravel Breeze ya proporciona un esqueleto básico de autenticación, por lo que puedes extenderlo para incluir tu lógica de gestión de roles y permisos.

3.7 Conclusión

Aunque Laravel Breeze proporciona una excelente base para la autenticación, la implementación de un sistema de gestión de roles y permisos requiere pasos adicionales. Integrar este sistema en tu aplicación te permitirá controlar el acceso a diferentes partes de tu aplicación basándote en los roles y permisos de los usuarios. Recuerda siempre proteger adecuadamente las rutas y ofrecer una gestión segura de roles y permisos desde tu interfaz de usuario.